

Classes IOS – Swift

Métodos

Parte 2

X-Code

Prof. Agesandro Scarpioni
agesandro@fiap.com.br

Classes – Métodos

- No Swift, criar métodos com muitos parâmetros e passar estes parâmetros para os métodos é bem simples. Vamos ver 4 exemplos sem o return:

- Java

- `public void CalcularImcComPesoeAltura(float peso, float altura) { código aqui };`

- VB

- `Public Sub CalcularImcComPesoeAltura(Byval peso as Single, Byval altura as Single) (código aqui) End Sub`

- Objective-C

- `- (void) calcularImcComPeso:(float) peso eAltura:(float) altura { código aqui }`

- Swift

- `func calcularImc(peso:Float, altura:Float) { código aqui }`

Classes – Métodos

- Em nosso exemplo de classes no Swift o método tem o nome `calcularImc`, veja que o nome foi adaptado para receber dois parâmetros, e não há indicação de retorno pois se trata de uma sub rotina, mesmo com a palavra `func` no início do método. Veja a chamada abaixo:
- `a.calcularImc(peso: 100 Altura: 1.85)`

Classes – Métodos

- Vamos implementar uma sub rotina e uma function na classe Atleta.swift, lembrando que em POO esses dois são conhecidos como “métodos”

```
30     func calcularIMC(peso:Float, altura:Float){
31         let imc = peso / (altura * altura)
32         let formatado = String(format: "%0.2f", imc)
33         print("O IMC de \(self.nome) é \(formatado)")
34     }
35
36     func calcularIMC_2(peso:Float, altura:Float)->Float{
37         //return peso / (altura * altura)
38         //ou
39         return peso / pow(altura,2)
40     }
```

DICAS: Caso prefira use pow para trabalhar com potência → $imc = peso / pow(altura, 2)$, também é possível formatar, veja o exemplo na sub rotina formatada com 2 casas decimais.

Classes – Métodos

- No Main vamos chamar os 2 métodos criados, calcularIMC (sub rotina) e calcularIMC_2 (function)

```
Classe Exemplo > Classe Exemplo > main.swift > No Selection
fica claro que voce nao esta mais usando a instancia
Pessoa:
51 referencia3=nil
52
53 //chamando uma Sub-rotina que exibe um texto
54 a2.calcularIMC(peso: 100, altura: 1.85)
55
56 //imprimindo os dados de uma função que retorna um Float
57 print(a2.calcularIMC_2(peso: 80, altura: 1.75))
58
```

Caio está sendo desinicializado
Marina está sendo inicializada
Marina está sendo desinicializada
O IMC de Carlos é 29.22
26.12245
Program ended with exit code: 0

Métodos de Instância

- Foi criado nos slides anteriores dois métodos de instância, criaremos mais um, só que desta vez será uma função sem parâmetro que retorna um texto. Vá na classe `Atleta.swift` e digite o método abaixo:

```
42     func exibirAtleta() ->String{  
43         return "O Atleta é " + self.nome  
44     }
```

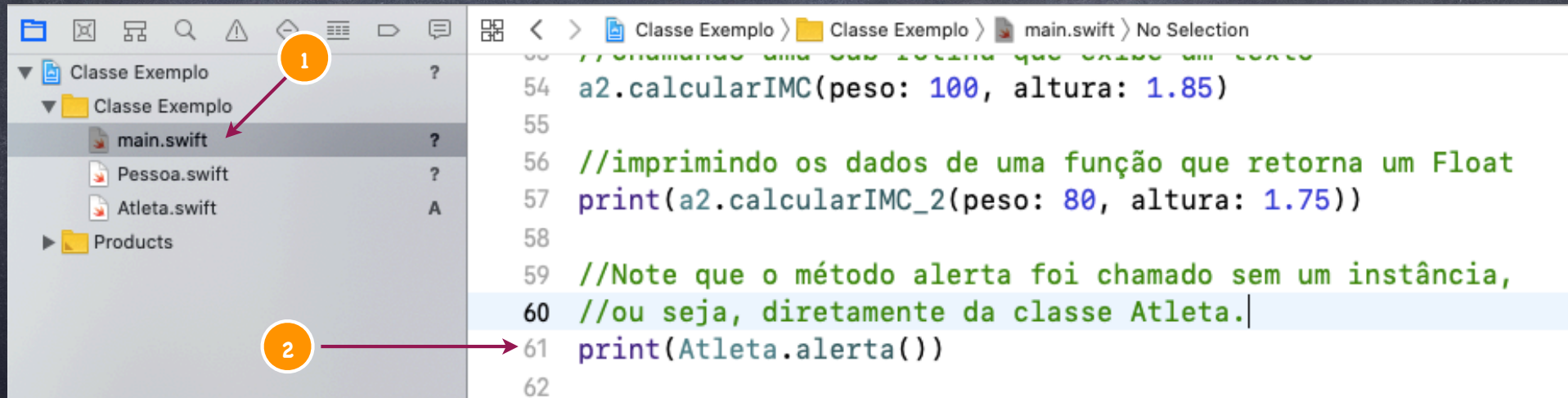
Métodos de Classe

- O método de Classe funciona como o método estático do Java, porém, ao invés de iniciar com a palavra func da página anterior o nome do método é iniciado com a palavra class.

```
49  
50     class func alerta() -> String{  
51         return "Me preparando a competição irá iniciar em breve"  
52     }
```

Chamando o método estático

- No Main (1) faça a programação da linha 61 (2). Para chamar um método estático você não precisa instanciar um objeto, basta chamar o método seguido do nome da classe.



Criando um método com parâmetro opcional

1

```
54  
55 func inscreverNaCompetição(modalidade:String, cidade:String? = nil)->String{  
56     if cidade == nil{  
57         return "Inscrito na competição para \((modalidade))"  
58     }else{  
59         return "Inscrito na competição para \((modalidade)) em \((cidade!))"  
60     }  
61 }
```

Chamando um método com parâmetro opcional

```

26
27 let resposta = a2.inscreverNaCompetição(modalidade: "Corrida de Aventura", cidade: nil)
28 print(resposta)
29
30 let resposta2 = a2.inscreverNaCompetição(modalidade: "Iron Man", cidade: "Florianópolis")
31 print(resposta2)
32
33

```

Line: 22 Col: 1

```

-----
Inscrito na competição para Corrida de Aventura
Inscrito na competição para Iron Man em Florianópolis
Program ended with exit code: 0

```

- Também é possível chamar o método sem a necessidade de atribuir "nil" na cidade conforme o exemplo abaixo:

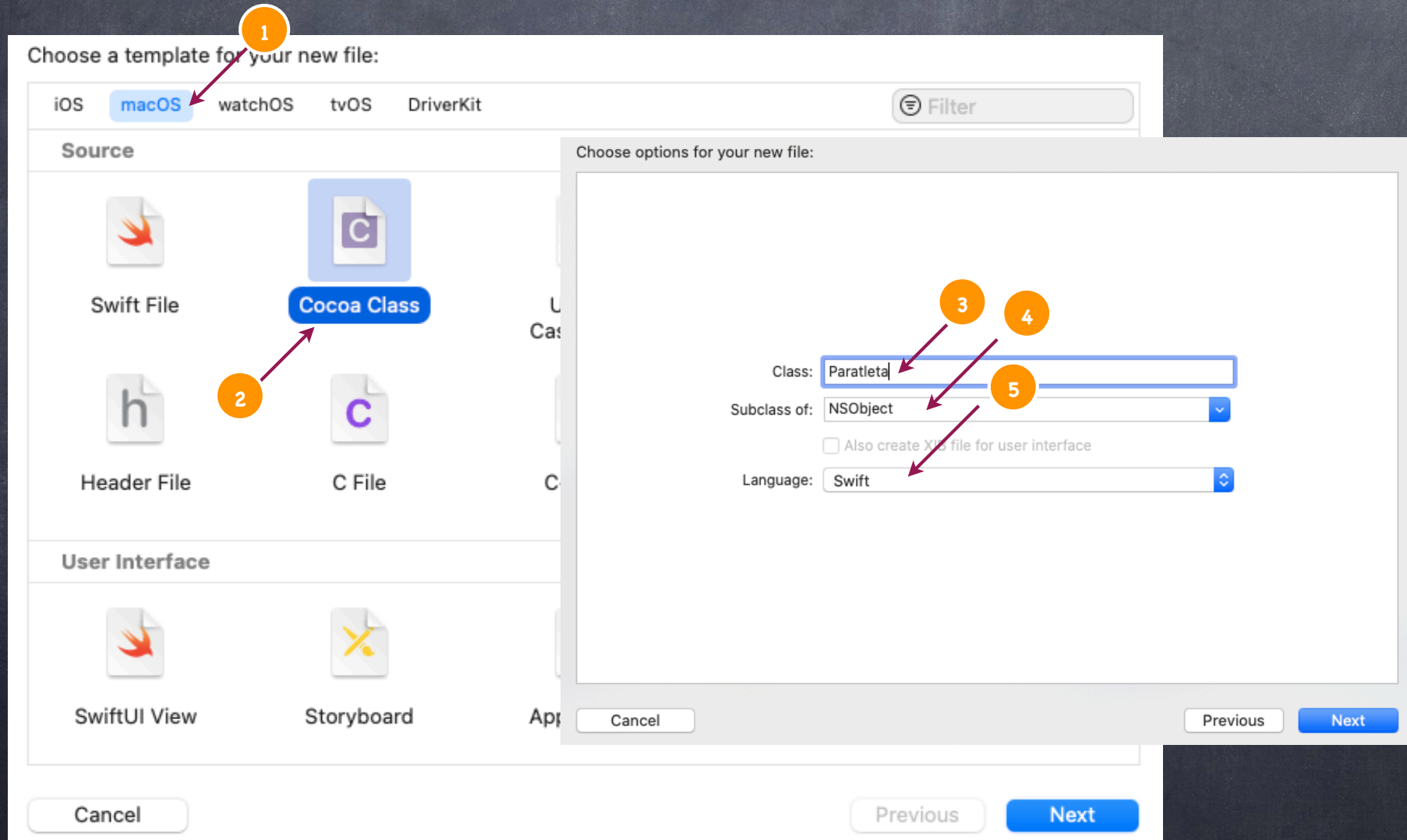
```

47
48 print(a.inscreverNaCompeticao(modalidade: "Natação"))
49

```

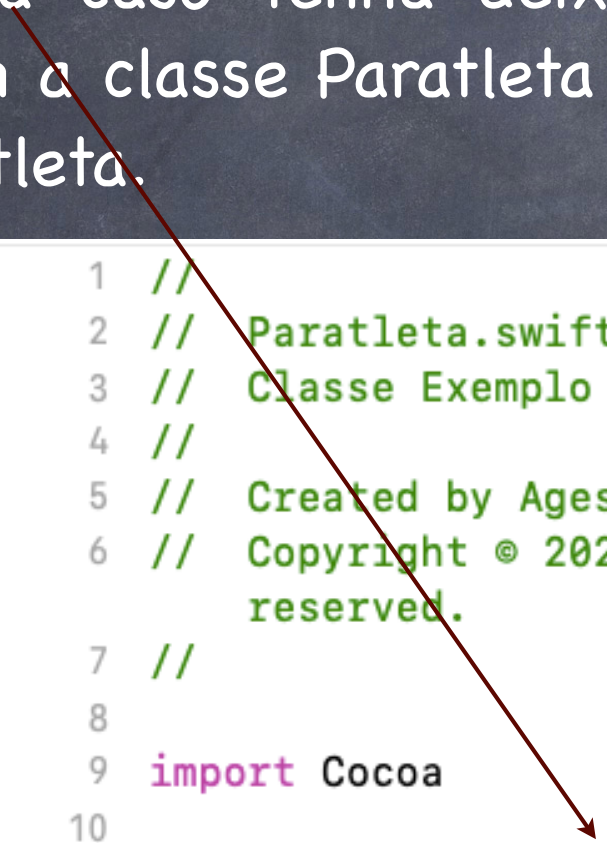
Herança

- ❶ Criar a classe Paratleta(3) subclasse de NSObject ou já digite "Atleta" em (4). Clique em File → New File → macOS (1) → double clique em Cocoa Class (2), ou, Command + N.



- Na classe Paratleta, altere a subclasse NSObject para Atleta caso tenha deixado NSObject na tela anterior, assim a classe Paratleta irá herdar os atributos e métodos de Atleta.

```
1 //  
2 // Paratleta.swift  
3 // Classe Exemplo  
4 //  
5 // Created by Agesandro Scarpioni on 17/02/20.  
6 // Copyright © 2020 Agesandro Scarpioni. All rights  
  reserved.  
7 //  
8  
9 import Cocoa  
10  
11 class Paratleta: Atleta {  
12  
13 }
```



Herança – Sobreposição

- Crie o atributo sem acento “deficiencia”, em Paratleta e ja crie um construtor padrão.

```
1 //  
2 // Paratleta.swift  
3 // Classe Exemplo  
4 //  
5 // Created by Agesandro Scarpioni on 17/02/20.  
6 // Copyright © 2020 Agesandro Scarpioni. All rights  
  reserved.  
7 //  
8  
9 import Cocoa  
10  
11 class Paratleta: Atleta {  
12  
13     var deficiencia:String  
14  
15     override init() {  
16         deficiencia=""  
17         super.init()  
18     }  
19 }
```

Herança – Sobreposição

- Crie um método `exibirAtleta` na classe `Paratleta` que retorne uma string, use `override` pois esse método será sobreposto ao método `exibirAtleta` da classe `Atleta`.

```
1 //
2 // Paratleta.swift
3 // Classe Exemplo
4 //
5 // Created by Agesandro Scarpioni on 17/02/20.
6 // Copyright © 2020 Agesandro Scarpioni. All rights
  reserved.
7 //
8
9 import Cocoa
10
11 class Paratleta: Atleta {
12
13     var deficiencia:String
14
15     override init() {
16         deficiencia=""
17         super.init()
18     }
19
20     override func exibirAtleta() -> String {
21         code
22     }
23 }
24
```

Herança – Sobreposição

- Na implementação da linha 21, observe que estamos invocando o método `exibirAtleta` da classe `Atleta`, por isso usamos `super` e completamos com dados da classe `ParaAtleta`.

```
1 //
2 // Paratleta.swift
3 // Classe Exemplo
4 //
5 // Created by Agesandro Scarpioni on 17/02/20.
6 // Copyright © 2020 Agesandro Scarpioni. All rights reserved.
7 //
8
9 import Cocoa
10
11 class Paratleta: Atleta {
12
13     var deficiencia:String
14
15     override init() {
16         deficiencia=""
17         super.init()
18     }
19
20     override func exibirAtleta() -> String {
21         return super.exibirAtleta() + " deficiência relatada \"(deficiencia)\"
22     }
23 }
```

Herança

- No main (1), crie um novo objeto para Paratleta (2), faça a carga de algumas informações nos atributos (3) e chame o método da classe Paratleta (4).

```
53 //chamando uma Sub-rotina que exibe um texto
54 a2.calcularIMC(peso: 100, altura: 1.85)
55
56 //imprimindo os dados de uma função que retorna um Float
57 print(a2.calcularIMC_2(peso: 80, altura: 1.75))
58
59 //Note que o método alerta foi chamado sem um instância,
60 //ou seja, diretamente da classe Atleta.
61 print(Atleta.alerta())
62
63 let p = Paratleta()
64 p.nome = "Jonas"
65 p.idade = 21
66 p.deficiencia = "Braço direito amputado"
67 print(p.exibirAtleta())
68
```

Nome: Ana
Idade: 22
Atleta Ana tem 22 anos
Atleta Carlos tem 26 anos
Atleta Caio tem 45 anos
Caio está sendo desinicializado
Marina está sendo inicializada
Marina está sendo desinicializada
O IMC de Carlos é 29.22
26.12245
A competição irá iniciar em 10 minutos
O Atleta é Jonas deficiência relatada Braço direito amputado
Program ended with exit code: 0

Métodos – Sobrecarga

- Para utilizar sobrecarga em Swift basta incluir novos parâmetros e manter o mesmo nome da função, vale tanto para métodos de classe como o exemplo abaixo, como para métodos de instância, veja o exemplo na classe Atleta:

```
40     class func alerta() -> String{  
41         return "A competição irá iniciar em breve"  
42     }  
43  
44     class func alerta(tempoEmMinutos:Int)->String{  
45         return "A competição ira iniciar em \(tempoEmMinutos) minutos"  
46     }
```

- E a chamada dos métodos no Main:

```
68  
69 //exemplo de sobrecarga (Overload)  
70 print(Atleta.alerta())  
71 print(Atleta.alerta(tempoEmMinutos: 30))  
72  
73
```

Comparação entre Objetos

- Apesar das duas classes terem os mesmos dados elas não são iguais, são objetos diferentes, veja o playground abaixo:

```
1 import UIKit
2
3 var str = "Hello, playground"
4
5 class Atleta{
6
7     var nome:String
8     var idade:Int
9
10    init(){
11        self.idade = 0
12        self.nome = ""
13    }
14    init(nome:String, idade:Int){
15        self.nome = nome
16        self.idade = idade
17    }
18    deinit {
19        print("\(nome) está sendo desinicializado")
20    }
21 }
22
23 let umAtleta = Atleta(nome: "Carlos", idade: 26)
24 let outroAtleta = Atleta(nome: "Carlos", idade: 26)
25
26 if umAtleta === outroAtleta{
27     print("São iguais")
28 }else{
29     print("Não são iguais")
30 }
31
```

"Hello, playground"

Atleta
Atleta

1

"Não são iguais\n"

Não são iguais

Comparação entre Objetos

- Continuando o exemplo de comparação uso de referências para o teste2 e teste3 que são iguais.

23	let umAtleta = Atleta(nome: "Carlos", idade: 26)	Atleta
24	let outroAtleta = Atleta(nome: "Carlos", idade: 26)	Atleta
25	if umAtleta === outroAtleta{	
26	print("São iguais")	
27	}else{	
28	print("Não são iguais")	"Não são iguais\n"
29	}	
30		
31	var teste1:Atleta?	nil
32	var teste2:Atleta?	nil
33	var teste3:Atleta?	nil
34	var teste4:Atleta?	nil
35		
36	teste1 = Atleta(nome: "Maria", idade: 29)	Atleta
37	teste2 = Atleta(nome: "Maria", idade: 29)	Atleta
38	teste3 = teste2	Atleta
39	teste4 = Atleta(nome: "Jorge", idade: 31)	Atleta
40	if teste1 === teste2{	
41	print("São iguais")	
42	}else{	
43	print("Não são iguais")	"Não são iguais\n"
44	}	
45	if teste2 === teste3{	
46	print("São iguais")	"São iguais\n"
47	}else{	
48	print("Não são iguais")	
49	}	
50	if teste3 === teste4{	
51	print("São iguais")	
52	}else{	
53	print("Não são iguais")	"Não são iguais\n"
	}	
Não são iguais Não são iguais São iguais Não são iguais		

Prática

Criação de um programa para testarmos todos os conceitos deste tópico.

- Ainda no projeto da classe Enfermeira criado anteriormente, desenvolva um método do tipo void e três métodos que retornem respectivamente um bool , um String e um Int.
- Como dica, vocês podem criar um método que entre com a temperatura do corpo e retorne um booleano para indicar que o paciente está com febre, ou ainda um método que receba dois parâmetros e retorne a temperatura da água para um banho em graus celsius. Faça uso do método construtor com pelo menos 3 atributos.